

Guide des Interfaces STM32 avec STM32CubeIDE

Table des Matières

1. [Les interfaces d'E/S numériques \(GPIO\)](#)
 2. [Les interfaces Timers/Counters](#)
 3. [Le convertisseur analogique numérique \(ADC\)](#)
 4. [Le convertisseur numérique analogique \(DAC\)](#)
 5. [Interface USART](#)
 6. [Interface I2C](#)
 7. [Interface SPI](#)
 8. [Bus CAN](#)
-

1. Les interfaces d'E/S numériques (GPIO)

Introduction

Les GPIO (General Purpose Input/Output) sont les broches d'entrée/sortie numériques du microcontrôleur STM32. Elles permettent d'interfacer le microcontrôleur avec le monde extérieur.

Caractéristiques principales

- **Modes de fonctionnement** : Input, Output, Alternate Function, Analog
- **Vitesse** : Low, Medium, High, Very High
- **Configuration** : Pull-up, Pull-down, No pull
- **Type de sortie** : Push-pull, Open-drain

Configuration avec CubeIDE

Configuration graphique (CubeMX)

1. Pinout & Configuration → System Core → GPIO

2. Cliquer sur la broche désirée dans le schéma

3. Sélectionner le mode : ou

4. Configurer les paramètres :

- **GPIO mode** : Input/Output
- **GPIO Pull-up/Pull-down** : No pull/Pull-up/Pull-down
- **Maximum output speed** : Low/Medium/High/Very High
- **GPIO output level** : High/Low (pour les sorties)

Code généré

```
c

// Initialisation automatique dans main.c
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /* Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);

    /* Configure GPIO pin : PA5 */
    GPIO_InitStruct.Pin = GPIO_PIN_5;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}
```

Fonctions principales HAL

c

```
// Lecture d'une entrée
GPIO_PinState state = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0);

// Écriture sur une sortie
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);

// Basculement d'une sortie
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
```

Exemple pratique : Clignotement LED

c

```
while (1)
{
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    HAL_Delay(500);
}
```

2. Les interfaces Timers/Counters

Introduction

Les timers STM32 sont des périphériques polyvalents permettant de mesurer le temps, générer des signaux PWM, compter des événements, et déclencher des interruptions.

Types de timers

- **Basic timers** (TIM6, TIM7) : Comptage simple
- **General-purpose timers** (TIM2-TIM5) : PWM, Input Capture, Output Compare
- **Advanced timers** (TIM1, TIM8) : Contrôle moteur, temps mort

Configuration avec CubeIDE

Timer en mode compteur

1. **Pinout & Configuration** → **Timers** → **TIM2**
2. **Clock Source** : Internal Clock
3. **Prescaler** : Valeur pour diviser la fréquence
4. **Counter Period** : Valeur max du compteur
5. **Counter Mode** : Up/Down/Center Aligned

Calcul de la période

$$\text{Période} = (\text{Prescaler} + 1) \times (\text{Counter Period} + 1) / \text{Fréquence_horloge}$$

Fonctions principales HAL

```
c
// Démarrage du timer
HAL_TIM_Base_Start(&htim2);

// Démarrage avec interruption
HAL_TIM_Base_Start_IT(&htim2);

// Arrêt du timer
HAL_TIM_Base_Stop(&htim2);

// Lecture de la valeur du compteur
uint32_t counter = __HAL_TIM_GET_COUNTER(&htim2);
```

PWM (Pulse Width Modulation)

Configuration PWM

1. **Timer** → **Channel1** → **PWM Generation CH1**
2. **Mode** : PWM mode 1
3. **Pulse** : Valeur du rapport cyclique

Code PWM

```
c
// Démarrage PWM
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

// Modification du rapport cyclique
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 500);
```


Input Capture

```
c

// Démarrage Input Capture
HAL_TIM_IC_Start_IT(&htim2, TIM_CHANNEL_1);

// Callback d'interruption
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
    {
        uint32_t capture = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
    }
}
```

3. Le convertisseur analogique numérique (ADC)

Introduction

L'ADC convertit des signaux analogiques en valeurs numériques. Les STM32 disposent d'ADC 12 bits avec plusieurs canaux d'entrée.

Caractéristiques

- **Résolution** : 12 bits (0-4095)
- **Tension de référence** : VDD (3.3V typique)
- **Temps de conversion** : Configurable
- **Modes** : Single, Continuous, Scan

Configuration avec CubeIDE

Configuration de base

1. **Pinout & Configuration** → **Analog** → **ADC1**
2. **IN0** : Sélectionner le canal (ex: PA0)
3. **Configuration** :
 - **Clock Prescaler** : Asynchronous clock mode
 - **Resolution** : 12 bits
 - **Data Alignment** : Right aligned
 - **Scan Conversion Mode** : Disabled/Enabled
 - **Continuous Conversion Mode** : Disabled/Enabled

Paramètres du canal

- **Sampling Time** : 3 Cycles à 640.5 Cycles
- **Rank** : Ordre de conversion (pour mode scan)

Fonctions principales HAL

Conversion simple

```
c

// Calibration ADC
HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);

// Démarrage conversion
HAL_ADC_Start(&hadc1);

// Attente fin de conversion
HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);

// Lecture de la valeur
uint32_t adcValue = HAL_ADC_GetValue(&hadc1);

// Conversion en tension
float voltage = (adcValue * 3.3f) / 4095.0f;
```

Conversion continue avec interruption

```
c

// Démarrage conversion continue avec IT
HAL_ADC_Start_IT(&hadc1);

// Callback de fin de conversion
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance == ADC1)
    {
        uint32_t adcValue = HAL_ADC_GetValue(&hadc1);
        // Traitement de la valeur
    }
}
```

Mode DMA

c

```
uint32_t adcBuffer[100];

// Démarrage ADC avec DMA
HAL_ADC_Start_DMA(&hadc1, adcBuffer, 100);

// Callback DMA complete
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    // Traitement du buffer complet
}
```

4. Le convertisseur numérique analogique (DAC)

Introduction

Le DAC convertit des valeurs numériques en signaux analogiques. Utile pour générer des signaux de référence, des formes d'onde, ou commander des circuits analogiques.

Caractéristiques

- **Résolution** : 12 bits
- **Tension de sortie** : 0 à VDD
- **Modes** : Normal, Buffer enabled
- **Déclenchement** : Software, Timer, External

Configuration avec CubeIDE

Configuration de base

1. **Pinout & Configuration** → **Analog** → **DAC1**
2. **OUT1** : Activer la sortie (PA4)
3. **Configuration** :
 - **Trigger** : None (Software trigger)
 - **Output Buffer** : Enabled/Disabled
 - **Connected to external pin and to on chip peripherals**

Fonctions principales HAL

c

```
// Démarrage du DAC
HAL_DAC_Start(&hdac1, DAC_CHANNEL_1);

// Écriture d'une valeur (12 bits)
HAL_DAC_SetValue(&hdac1, DAC_CHANNEL_1, DAC_ALIGN_12B_R, 2048);

// Conversion en tension
// Tension = (valeur * VDD) / 4095
float voltage = 1.65f; // Exemple : 1.65V
uint32_t dacValue = (voltage * 4095.0f) / 3.3f;
HAL_DAC_SetValue(&hdac1, DAC_CHANNEL_1, DAC_ALIGN_12B_R, dacValue);
```

💡 Génération de forme d'onde

c

```
// Table de sinus (64 points)
const uint16_t sineWave[64] = {
    2048, 2249, 2447, 2642, 2831, 3013, 3185, 3346,
    3495, 3630, 3750, 3853, 3939, 4007, 4056, 4085,
    4095, 4085, 4056, 4007, 3939, 3853, 3750, 3630,
    // ... reste de la table
};

// Génération continue avec timer
void generateSineWave(void)
{
    static uint8_t index = 0;
    HAL_DAC_SetValue(&hdac1, DAC_CHANNEL_1, DAC_ALIGN_12B_R, sineWave[index]);
    index = (index + 1) % 64;
}
```

5. Interface USART

🎯 Introduction

L'USART (Universal Synchronous/Asynchronous Receiver/Transmitter) permet la communication série avec d'autres dispositifs ou un PC.

📌 Caractéristiques

- **Modes** : Asynchrone, Synchrone, Half-duplex
- **Vitesse** : Configurable (9600, 115200 bauds typiques)
- **Format** : 7, 8, 9 bits de données
- **Parité** : None, Even, Odd
- **Stop bits** : 1, 1.5, 2

Configuration avec CubelIDE

Configuration de base

1. **Pinout & Configuration** → **Connectivity** → **USART2**
2. **Mode** : Asynchronous
3. **Configuration** :
 - **Baud Rate** : 115200 Bits/s
 - **Word Length** : 8 Bits
 - **Parity** : None
 - **Stop Bits** : 1
 - **Data Direction** : Receive and Transmit

Fonctions principales HAL

Transmission/Réception basique

```
c

// Transmission de données
char message[] = "Hello World!\r\n";
HAL_UART_Transmit(&huart2, (uint8_t*)message, strlen(message), HAL_MAX_DELAY);

// Réception de données
uint8_t rxBuffer[100];
HAL_UART_Receive(&huart2, rxBuffer, 10, HAL_MAX_DELAY);
```

Mode interruption

```

c
uint8_t rxData;

// Réception avec interruption
HAL_UART_Receive_IT(&huart2, &rxData, 1);

// Callback de réception
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART2)
    {
        // Traitement de rxData
        HAL_UART_Transmit_IT(&huart2, &rxData, 1); // Echo
        HAL_UART_Receive_IT(&huart2, &rxData, 1); // Nouvelle réception
    }
}

```

Mode DMA

```

c
uint8_t txBuffer[100];
uint8_t rxBuffer[100];

// Transmission DMA
HAL_UART_Transmit_DMA(&huart2, txBuffer, 100);

// Réception DMA
HAL_UART_Receive_DMA(&huart2, rxBuffer, 100);

```

Exemple : Communication avec PC

```

c
// Fonction d'envoi formaté
int _write(int file, char *ptr, int len)
{
    HAL_UART_Transmit(&huart2, (uint8_t*)ptr, len, HAL_MAX_DELAY);
    return len;
}

// Utilisation avec printf
printf("Valeur ADC: %d\r\n", adcValue);

```

6. Interface I2C

Introduction

L'I2C (Inter-Integrated Circuit) est un bus série bidirectionnel utilisant seulement 2 fils : SDA (données) et SCL (horloge).

Caractéristiques

- **Topologie** : Multi-maître, multi-esclave
- **Vitesse** : Standard (100 kHz), Fast (400 kHz), Fast+ (1 MHz)
- **Adressage** : 7 ou 10 bits
- **Fils** : SDA, SCL + masse

Configuration avec CubeIDE

Configuration de base

1. **Pinout & Configuration** → **Connectivity** → **I2C1**
2. **Mode** : I2C
3. **Configuration** :
 - **I2C Speed Mode** : Standard Mode
 - **I2C Clock Speed** : 100000 Hz
 - **Clock No Stretch Mode** : Disabled
 - **General Call Address Detection** : Disabled

Fonctions principales HAL

Communication basique

```

c

#define DEVICE_ADDR 0x68 << 1 // Adresse 7 bits décalée

// Transmission
uint8_t data[] = {0x01, 0x02, 0x03};
HAL_I2C_Master_Transmit(&hi2c1, DEVICE_ADDR, data, 3, HAL_MAX_DELAY);

// Réception
uint8_t buffer[10];
HAL_I2C_Master_Receive(&hi2c1, DEVICE_ADDR, buffer, 10, HAL_MAX_DELAY);

// Écriture dans un registre
uint8_t reg = 0x20;
uint8_t value = 0xFF;
HAL_I2C_Mem_Write(&hi2c1, DEVICE_ADDR, reg, I2C_MEMADD_SIZE_8BIT, &value, 1, HAL_MAX_DELAY);

// Lecture d'un registre
uint8_t readValue;
HAL_I2C_Mem_Read(&hi2c1, DEVICE_ADDR, reg, I2C_MEMADD_SIZE_8BIT, &readValue, 1, HAL_MAX_DELAY);

```

Détection de périphériques

```

c

// Scanner les adresses I2C
for(uint8_t addr = 1; addr < 128; addr++)
{
    if(HAL_I2C_IsDeviceReady(&hi2c1, addr << 1, 1, 1000) == HAL_OK)
    {
        printf("Device found at address: 0x%02X\r\n", addr);
    }
}

```

💡 Exemple : Lecture capteur de température DS1621

c

```
#define DS1621_ADDR 0x48 << 1
#define READ_TEMP_CMD 0xAA

float readTemperature(void)
{
    uint8_t cmd = READ_TEMP_CMD;
    uint8_t data[2];

    // Envoi commande
    HAL_I2C_Master_Transmit(&hi2c1, DS1621_ADDR, &cmd, 1, HAL_MAX_DELAY);

    // Lecture résultat
    HAL_I2C_Master_Receive(&hi2c1, DS1621_ADDR, data, 2, HAL_MAX_DELAY);

    // Conversion
    int16_t temp = (data[0] << 8) | data[1];
    return temp / 256.0f;
}
```

7. Interface SPI

Introduction

Le SPI (Serial Peripheral Interface) est un protocole de communication série synchrone full-duplex utilisant 4 fils.

Caractéristiques

- **Fils** : MOSI, MISO, SCK, CS (NSS)
- **Topologie** : Maître-esclave
- **Vitesse** : Jusqu'à plusieurs MHz
- **Mode** : Full-duplex, Half-duplex, Simplex

Configuration avec CubeIDE

Configuration maître

1. Pinout & Configuration → Connectivity → SPI1

2. **Mode** : Full-Duplex Master

3. **Configuration** :

- **Prescaler** : Divise l'horloge système
- **Clock Polarity (CPOL)** : Low/High
- **Clock Phase (CPHA)** : 1 Edge/2 Edge
- **Data Size** : 8 Bits
- **First Bit** : MSB First

Modes SPI

Mode	CPOL	CPHA	Description
0	0	0	Horloge idle à 0, échantillonnage front montant
1	0	1	Horloge idle à 0, échantillonnage front descendant
2	1	0	Horloge idle à 1, échantillonnage front descendant
3	1	1	Horloge idle à 1, échantillonnage front montant

Fonctions principales HAL

```
c

// Transmission
uint8_t txData[] = {0x01, 0x02, 0x03};
HAL_SPI_Transmit(&hspi1, txData, 3, HAL_MAX_DELAY);

// Réception
uint8_t rxData[10];
HAL_SPI_Receive(&hspi1, rxData, 10, HAL_MAX_DELAY);

// Transmission/Réception simultanée
uint8_t txBuffer[5] = {0x01, 0x02, 0x03, 0x04, 0x05};
uint8_t rxBuffer[5];
HAL_SPI_TransmitReceive(&hspi1, txBuffer, rxBuffer, 5, HAL_MAX_DELAY);
```

Exemple : Communication avec un écran LCD

c

```
#define LCD_CS_Pin GPIO_PIN_4
#define LCD_CS_Port GPIOA

void LCD_WriteCommand(uint8_t cmd)
{
    HAL_GPIO_WritePin(LCD_CS_Port, LCD_CS_Pin, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, &cmd, 1, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(LCD_CS_Port, LCD_CS_Pin, GPIO_PIN_SET);
}

void LCD_WriteData(uint8_t data)
{
    HAL_GPIO_WritePin(LCD_CS_Port, LCD_CS_Pin, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, &data, 1, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(LCD_CS_Port, LCD_CS_Pin, GPIO_PIN_SET);
}
```

8. Bus CAN



Introduction

Le bus CAN (Controller Area Network) est un protocole de communication robuste utilisé principalement dans l'automobile et l'industrie.



Caractéristiques

- **Topologie** : Bus différentiel (CAN_H, CAN_L)
- **Vitesse** : 125 kbps à 1 Mbps
- **Priorité** : Messages priorisés par identifiant
- **Robustesse** : Détection et correction d'erreurs



Configuration avec CubeIDE

Configuration de base

1. Pinout & Configuration → Connectivity → CAN1

2. **Mode** : Activated

3. **Configuration** :

- **Prescaler** : Dépend de la vitesse désirée
- **Time Quanta in Bit Segment 1** : 13
- **Time Quanta in Bit Segment 2** : 2
- **ReSynchronization Jump Width** : 1

Calcul de la vitesse

$$\text{Vitesse} = f_{\text{CAN}} / (\text{Prescaler} \times (1 + \text{BS1} + \text{BS2}))$$

Configuration des filtres

```
c
// Configuration du filtre CAN
CAN_FilterTypeDef canFilterConfig;
canFilterConfig.FilterActivation = ENABLE;
canFilterConfig.FilterBank = 0;
canFilterConfig.FilterFIFOAssignment = CAN_RX_FIFO0;
canFilterConfig.FilterIdHigh = 0x0000;
canFilterConfig.FilterIdLow = 0x0000;
canFilterConfig.FilterMaskIdHigh = 0x0000;
canFilterConfig.FilterMaskIdLow = 0x0000;
canFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;
canFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;

HAL_CAN_ConfigFilter(&hcan1, &canFilterConfig);
```

Fonctions principales HAL

Démarrage CAN

```
c
// Démarrage du CAN
HAL_CAN_Start(&hcan1);

// Activation des notifications
HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING);
```

Transmission

c

```
CAN_TxHeaderTypeDef txHeader;
uint32_t txMailbox;
uint8_t txData[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};

// Configuration de l'en-tête
txHeader.StdId = 0x123;
txHeader.ExtId = 0;
txHeader.RTR = CAN_RTR_DATA;
txHeader.IDE = CAN_ID_STD;
txHeader.DLC = 8;
txHeader.TransmitGlobalTime = DISABLE;

// Envoi du message
HAL_CAN_AddTxMessage(&hcan1, &txHeader, txData, &txMailbox);
```

Réception

c

```
CAN_RxHeaderTypeDef rxHeader;
uint8_t rxData[8];

// Callback de réception
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
{
    if(HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &rxHeader, rxData) == HAL_OK)
    {
        // Traitement du message reçu
        printf("ID: 0x%03X, Data: ", rxHeader.StdId);
        for(int i = 0; i < rxHeader.DLC; i++)
        {
            printf("0x%02X ", rxData[i]);
        }
        printf("\r\n");
    }
}
```

Conseils pratiques pour STM32CubeIDE

Débogage

- Utiliser **Debug Configuration** pour le débogage en temps réel
- **Live Expressions** pour surveiller les variables
- **SWV (Serial Wire Viewer)** pour les traces en temps réel

⚡ Optimisation

- Activer uniquement les périphériques nécessaires
- Choisir la fréquence d'horloge appropriée
- Utiliser le mode **Low Power** quand possible

📝 Bonnes pratiques

- Toujours vérifier les valeurs de retour HAL
- Utiliser les callbacks pour les traitements asynchrones
- Gérer les timeout appropriés
- Documenter les configurations matérielles

🔑 Configuration des horloges

```
c

// Exemple de configuration RCC
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

    /** Initializes the RCC Oscillators according to the specified parameters */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 8;
    RCC_OscInitStruct.PLL.PLLN = 360;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
    RCC_OscInitStruct.PLL.PLLQ = 7;
    HAL_RCC_OscConfig(&RCC_OscInitStruct);
}
```



Exemples d'intégration avancés



Projet 1 : Station météorologique complète

```
c

// Structure de données météo
typedef struct {
    float temperature;
    float humidity;
    uint16_t light;
    float pressure;
} WeatherData_t;

WeatherData_t weather;

void readWeatherSensors(void)
{
    // Lecture température (ADC)
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    uint32_t tempRaw = HAL_ADC_GetValue(&hadc1);
    weather.temperature = ((tempRaw * 3.3f / 4095.0f) - 0.5f) * 100.0f;

    // Lecture capteur I2C (ex: BME280)
    uint8_t tempData[2];
    HAL_I2C_Mem_Read(&hi2c1, 0x76 << 1, 0xFA, 1, tempData, 2, HAL_MAX_DELAY);

    // Transmission données via USART
    printf("Temp: %.2f°C, Humidity: %.2f%%\r\n",
        weather.temperature, weather.humidity);
}
```



Projet 2 : Contrôleur moteur avec CAN

c

```
// Structure message CAN moteur
typedef struct {
    uint16_t speed;    // RPM
    uint8_t direction; // 0=CW, 1=CCW
    uint8_t enable;    // 0=OFF, 1=ON
} MotorControl_t;

void sendMotorCommand(MotorControl_t *motor)
{
    CAN_TxHeaderTypeDef txHeader;
    uint8_t txData[8];
    uint32_t txMailbox;

    // Configuration CAN
    txHeader.StdId = 0x200; // ID moteur
    txHeader.RTR = CAN_RTR_DATA;
    txHeader.IDE = CAN_ID_STD;
    txHeader.DLC = 4;

    // Préparation données
    txData[0] = motor->speed >> 8;
    txData[1] = motor->speed & 0xFF;
    txData[2] = motor->direction;
    txData[3] = motor->enable;

    // Envoi
    HAL_CAN_AddTxMessage(&hcan1, &txHeader, txData, &txMailbox);

    // Mise à jour PWM selon la vitesse
    uint32_t pwmValue = (motor->speed * 1000) / 3000; // Max 3000 RPM
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pwmValue);
}
```

💡 Projet 3 : Data Logger avec carte SD (SPI)

c

```
// Structure de log
typedef struct {
    uint32_t timestamp;
    float sensor1;
    float sensor2;
    uint16_t status;
} LogEntry_t;

void logDataToSD(LogEntry_t *entry)
{
    uint8_t buffer[16];

    // Sérialisation des données
    memcpy(&buffer[0], &entry->timestamp, 4);
    memcpy(&buffer[4], &entry->sensor1, 4);
    memcpy(&buffer[8], &entry->sensor2, 4);
    memcpy(&buffer[12], &entry->status, 2);

    // Écriture sur carte SD via SPI
    // (Code spécifique au protocole SD)
    HAL_SPI_Transmit(&hspi1, buffer, 16, HAL_MAX_DELAY);
}
```

Techniques avancées

Optimisation des performances

Utilisation du DMA

c

```
// Configuration DMA pour ADC multi-canal
uint32_t adcResults[4]; // 4 canaux ADC

void configureADC_DMA(void)
{
    // Configuration scan mode
    hadc1.Init.ScanConvMode = ADC_SCAN_ENABLE;
    hadc1.Init.ContinuousConvMode = ENABLE;
    HAL_ADC_Init(&hadc1);

    // Démarrage avec DMA circulaire
    HAL_ADC_Start_DMA(&hadc1, adcResults, 4);
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    // Traitement des 4 valeurs simultanément
    float voltage1 = (adcResults[0] * 3.3f) / 4095.0f;
    float voltage2 = (adcResults[1] * 3.3f) / 4095.0f;
    // ...
}
```

Communication non-bloquante

c

```
// État de la machine d'état UART
typedef enum {
    UART_IDLE,
    UART_TRANSMITTING,
    UART_RECEIVING
} UartState_t;

UartState_t uartState = UART_IDLE;

void handleUARTCommunication(void)
{
    switch(uartState)
    {
        case UART_IDLE:
            // Démarrer nouvelle opération
            break;
        case UART_TRANSMITTING:
            // Vérifier si transmission terminée
            break;
        case UART_RECEIVING:
            // Traiter données reçues
            break;
    }
}
```

Gestion d'erreurs robuste

c

```
// Fonction de vérification d'erreur universelle
HAL_StatusTypeDef checkHALStatus(HAL_StatusTypeDef status, const char* operation)
{
    if(status != HAL_OK)
    {
        printf("Erreur %s: %d\r\n", operation, status);
        // Log erreur ou action corrective
    }
    return status;
}

// Utilisation
if(checkHALStatus(HAL_I2C_Master_Transmit(&hi2c1, addr, data, len, timeout),
    "I2C Transmit") != HAL_OK)
{
    // Gestion d'erreur spécifique
    return ERROR_I2C_COMM;
}
```

Synchronisation multi-périphériques

c

```
// Synchronisation Timer + ADC + DMA
void startSynchronizedAcquisition(void)
{
    // Timer déclenche ADC à intervalles réguliers
    HAL_TIM_Base_Start(&htim3); // Timer trigger
    HAL_ADC_Start_DMA(&hadc1, adcBuffer, BUFFER_SIZE);

    // Timer déclenche conversion ADC
    // ADC stocke résultats via DMA
    // Callback DMA traite les données
}
```

Interfaces spécialisées

Interface USB (CDC)

c

```
// Configuration USB comme port série virtuel
void USB_CDC_Example(void)
{
    char message[100];
    sprintf(message, "STM32 USB CDC Test\r\n");

    // Transmission via USB
    CDC_Transmit_FS((uint8_t*)message, strlen(message));
}

// Réception USB
void CDC_ReceiveCallback(uint8_t* buffer, uint32_t length)
{
    // Echo des données reçues
    CDC_Transmit_FS(buffer, length);
}
```

Interface Ethernet (si disponible)

c

```
// Configuration basique Ethernet
void Ethernet_Init(void)
{
    // Configuration PHY
    // Configuration MAC
    // Configuration LwIP stack
}

// Serveur HTTP simple
void HTTP_Server_Task(void)
{
    // Écoute sur port 80
    // Réponse aux requêtes HTTP
}
```

Interface RF (SPI + module externe)

c

```
// Communication avec module RF (ex: nRF24L01)
void RF_SendPacket(uint8_t *data, uint8_t length)
{
    // Configuration registres via SPI
    uint8_t config[] = {0x20, 0x0E}; // Register + value
    HAL_GPIO_WritePin(RF_CS_Port, RF_CS_Pin, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, config, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(RF_CS_Port, RF_CS_Pin, GPIO_PIN_SET);

    // Envoi payload
    HAL_GPIO_WritePin(RF_CS_Port, RF_CS_Pin, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, data, length, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(RF_CS_Port, RF_CS_Pin, GPIO_PIN_SET);
}
```



Tests et validation



Tests unitaires des interfaces

c

```
// Test GPIO
HAL_StatusTypeDef testGPIO(void)
{
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
    HAL_Delay(10);

    if(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_5) == GPIO_PIN_SET)
        return HAL_OK;
    else
        return HAL_ERROR;
}

// Test loopback USART
HAL_StatusTypeDef testUART_Loopback(void)
{
    uint8_t testData = 0x55;
    uint8_t rxData;

    HAL_UART_Transmit(&huart2, &testData, 1, 1000);
    HAL_UART_Receive(&huart2, &rxData, 1, 1000);

    return (testData == rxData) ? HAL_OK : HAL_ERROR;
}
```



Monitoring en temps réel

c

```
// Structure de monitoring système
typedef struct {
    uint32_t cpuUsage;
    uint32_t freeHeap;
    uint32_t taskSwitches;
    float systemTemp;
} SystemMonitor_t;

void updateSystemMonitor(SystemMonitor_t *monitor)
{
    // Calcul usage CPU
    monitor->cpuUsage = calculateCPUUsage();

    // Température interne
    HAL_ADC_Start(&hadc_temp);
    HAL_ADC_PollForConversion(&hadc_temp, HAL_MAX_DELAY);
    uint32_t tempRaw = HAL_ADC_GetValue(&hadc_temp);
    monitor->systemTemp = convertToTemperature(tempRaw);

    // Envoi via monitoring interface
    sendMonitoringData(monitor);
}
```

Débogage avancé

Utilisation des breakpoints conditionnels

c

```
// Dans CubeIDE : clic droit sur breakpoint → Properties
// Condition : variable == specific_value
// Hit count : déclencher après N passages

int debugCounter = 0;
void debugFunction(void)
{
    debugCounter++; // Breakpoint conditionnel ici
    // Code à déboguer
}
```

Analyse des signaux avec SWV

c

```
// Configuration SWO pour traces
void SWO_Init(void)
{
    // Enable SWO trace
    // Dans CubeIDE : Debug Config → Debugger → Serial Wire Viewer
}

// Utilisation ITM pour traces
void ITM_SendString(char *str)
{
    while(*str)
    {
        ITM_SendChar(*str++);
    }
}
```

Watchdog et surveillance système

c

```
// Configuration Watchdog
void IWDG_Config(void)
{
    hiwdg.Instance = IWDG;
    hiwdg.Init.Prescaler = IWDG_PRESCALER_64;
    hiwdg.Init.Window = 4095;
    hiwdg.Init.Reload = 4095;
    HAL_IWDG_Init(&hiwdg);
}

// Rafraîchissement périodique
void systemHealthCheck(void)
{
    // Vérifications système
    if(systemOK())
    {
        HAL_IWDG_Refresh(&hiwdg); // Reset watchdog
    }
    // Si problème détecté, le watchdog réinitialisera le système
}
```

Documentation officielle

- **Reference Manual** : Description détaillée des registres
- **Programming Manual** : Guide de programmation Cortex-M
- **HAL User Manual** : Documentation de l'API HAL
- **CubeMX User Manual** : Guide d'utilisation de CubeMX

Outils de développement

- **STM32CubeIDE** : IDE intégré
- **STM32CubeMX** : Outil de configuration graphique
- **STM32CubeProgrammer** : Outil de programmation
- **STM32CubeMonitor** : Supervision en temps réel

Projets d'application

1. **Station météo** : ADC + capteurs + USART
2. **Système de contrôle moteur** : PWM + encodeur + CAN
3. **Data logger** : I2C + SPI + carte SD
4. **Interface HMI** : USART + écran + boutons

Références pour approfondir

- **Application Notes ST** : AN4013 (I2C), AN4750 (SPI), AN3116 (CAN)
- **STM32 HAL Library** : Documentation complète des fonctions
- **Community Forum** : STM32 Community pour support technique
- **GitHub Examples** : Exemples officiels STMicroelectronics

Dépannage courant

Problèmes fréquents et solutions

GPIO ne fonctionne pas

-  Vérifier activation horloge port (`(__HAL_RCC_GPIOx_CLK_ENABLE())`)
-  Contrôler configuration mode et vitesse
-  Vérifier connexions matérielles

I2C communication échoue

-  Vérifier résistances pull-up sur SDA/SCL (4.7kΩ typique)
-  Contrôler adresse périphérique (décalage 1 bit)
-  Utiliser `HAL_I2C_IsDeviceReady()` pour tester

SPI pas de réponse

-  Vérifier mode SPI (CPOL/CPHA) compatible avec périphérique
-  Contrôler vitesse d'horloge (pas trop rapide)
-  Gérer signal CS manuellement si nécessaire

ADC valeurs incorrectes

-  Effectuer calibration (`HAL_ADCEX_Calibration_Start()`)
-  Vérifier temps d'échantillonnage suffisant
-  Contrôler tension de référence

Timer ne se déclenche pas

-  Vérifier activation horloge timer
-  Calculer prescaler et période correctement
-  Activer interruptions dans NVIC si nécessaire

Configuration système avancée

Gestion de l'horloge système

c

```
// Configuration horloge haute performance
void SystemClock_Config_HighSpeed(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    // HSE + PLL pour fréquence maximale
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 8; // Division
    RCC_OscInitStruct.PLL.PLLN = 360; // Multiplication
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2; // Division finale

    HAL_RCC_OscConfig(&RCC_OscInitStruct);

    // Configuration bus système
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1; // 180 MHz
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4; // 45 MHz
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2; // 90 MHz

    HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5);
}
```

Gestion mémoire et RTOS

c

```
// Avec FreeRTOS (optionnel)
void createTasks(void)
{
    // Tâche acquisition capteurs
    xTaskCreate(sensorTask, "SENSOR", 128, NULL, 1, NULL);

    // Tâche communication
    xTaskCreate(commTask, "COMM", 128, NULL, 2, NULL);

    // Tâche contrôle
    xTaskCreate(controlTask, "CTRL", 128, NULL, 3, NULL);
}

void sensorTask(void *argument)
{
    while(1)
    {
        readAllSensors();
        vTaskDelay(pdMS_TO_TICKS(100)); // 100ms
    }
}
```

Architecture projet recommandée

Structure des fichiers

```
Src/
├── main.c          // Fonction principale
├── stm32f4xx_it.c  // Gestionnaires d'interruption
├── stm32f4xx_hal_msp.c // Configuration MSP
├── drivers/
│   ├── gpio_driver.c // Pilote GPIO personnalisé
│   ├── sensor_driver.c // Pilotes capteurs
│   └── comm_driver.c // Pilotes communication
├── applications/
│   ├── data_logger.c // Application logging
│   ├── motor_control.c // Contrôle moteur
│   └── user_interface.c // Interface utilisateur
└── utils/
    ├── math_utils.c // Fonctions mathématiques
    ├── string_utils.c // Manipulation chaînes
    └── debug_utils.c // Outils de débogage
```

Modularité du code

```
c

// Fichier sensor_interface.h
typedef struct {
    HAL_StatusTypeDef (*init)(void);
    HAL_StatusTypeDef (*read)(float *value);
    HAL_StatusTypeDef (*calibrate)(void);
} SensorInterface_t;

// Implémentation pour capteur I2C
SensorInterface_t tempSensor = {
    .init = tempSensor_Init,
    .read = tempSensor_Read,
    .calibrate = tempSensor_Calibrate
};

// Utilisation générique
float temperature;
if(tempSensor.read(&temperature) == HAL_OK)
{
    printf("Température: %.2f°C\r\n", temperature);
}
```

Ce guide complet vous donne tous les outils nécessaires pour maîtriser les interfaces STM32 avec CubeIDE. De la configuration de base aux techniques avancées, vous disposez maintenant d'une base solide pour vos projets embarqués.